

Beyond Compensation: Kernel-Level Deferral of Irreversible Effects for Autonomous AI Agents

The AgentTx Project Proposal

Author One

Author Two

Author Three

Author Four

Department of Computer Science and Engineering

[**TODO:** Institution] [**TODO:** Course code, B.Tech. Year III]

August 4, 2026

Abstract

Filesystem damage caused by autonomous coding agents is a solved problem. Production agents checkpoint every file-modifying action and restore on demand, and execute inside Landlock, seccomp and Seatbelt sandboxes by default. What remains unsolved — and is publicly conceded as *unsolvable* — is the outbound effect: the push, the web-hook, the email, the payment. Once a third party has observed an action, the prevailing view holds, no mechanism can take it back.

That view is correct but stated at the wrong boundary. Irreversibility begins at *observation*, not at the system call. Every deployed remedy — compensating transactions, sagas, idempotency keys — operates downstream of emission and is therefore best-effort by construction. Between the agent’s `sendmsg` and the packet leaving the machine lies a region nobody occupies: the kernel.

We propose AgentTx, which relocates the commit point into that region. Writes divert into a copy-on-write layer and outbound effects are captured in a kernel write-ahead log rather than emitted; commit applies both, abort discards both. For fire-and-forget effects this converts best-effort compensation into a hard guarantee. We do not reverse observation — we prevent it.

The mechanism, however, is only as meaningful as the policy governing it, and the governing question is *what constitutes a transaction for an agent*. Too short and the guarantee window is negligible; too long and the system holds an unbounded write-ahead log across an unbounded number of agent decisions, a regime the transactional-memory literature was never designed for. We argue for *verification-delimited* transactions: a transaction spans the interval over which no evidence about the agent’s work is available, and commits the moment evidence arrives.

AgentTx contributes (i) kernel-level deferral of fire-and-forget external effects; (ii) verification-delimited transaction boundaries, with the guarantee-window against staleness trade-off measured rather than asserted; (iii) an `int8`-quantised classifier evaluated *inside* an eBPF LSM hook; (iv) a four-class effect taxonomy

grounded in the literature on irrevocability, with the first measured distribution of real agent traces across it; and (v) the first head-to-head evaluation of agent rollback mechanisms under a single workload and metric set.

1 Introduction

A prompt injection is planted in the documentation of a widely used dependency. An agent reads it in the course of an ordinary task, and — acting with the user’s full authority — reads the user’s SSH private key and POSTs it to an external host. The POST is fire-and-forget: the agent does not require the response in order to continue.

Under every deployed protection this either succeeds, because the destination allowlist happened to permit the host, or because the approval prompt displayed a shell command whose system-call trace the user could not have predicted; or it is blocked by a static rule that somebody wrote in advance having anticipated this specific exfiltration path. Neither is a guarantee.

From the kernel’s perspective the agent is an ordinary process running under the user’s UID. From the user’s perspective it is a process whose behaviour is partly determined by text it read moments earlier — a confused deputy that is non-deterministic and ingests attacker-influenced input as a matter of routine operation.

1.1 What is already solved

We concede a great deal at the outset, because the value of this proposal depends on being precise about what remains.

Sandboxing is commodity. Codex CLI executes commands inside bubblewrap with unprivileged user namespaces, Landlock LSM filesystem filtering and seccomp-BPF syscall restriction on Linux, Seatbelt on macOS, and restricted tokens with synthetic SIDs on Windows [26]. Anthropic has open-sourced the isolation layer behind Claude Code’s sandbox tooling [17].

Filesystem rollback is commodity. Claude Code records a checkpoint of affected files before every filesystem-modifying action and exposes a rewind operation restoring any prior point in a session [11]. The research literature has converged on the same capability [5, 15].

Preventive egress control ships today. Codex applies default-deny egress with destination allowlists, performing DNS and IP classification before permitting a hostname [26]. ChatGPT Work exposes plan approval, configurable check-ins and per-action approval gates [9, 25].

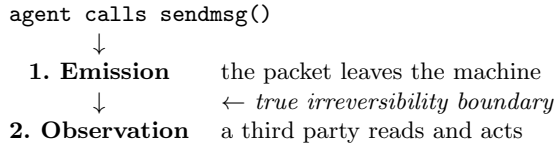
1.2 What is not solved

Practitioners now describe agent reversibility in three tiers: the filesystem, cheaply reversible through checkpoints and worktrees; the database, reversible only through explicitly coded compensating transactions; and external side effects — a sent email, a captured payment, a delivered webhook — described as *actions no mechanism can take back once a third party has seen them* [8].

That third tier is a stated impossibility claim, and on its own terms it is true. Observation transfers information, and no operation removes information from a party one does not control. A refund is not an un-charge; it is a second effect that happens to net to zero. Two-phase commit works only because participants agree in advance to prepare; a third party who never joined the protocol offers no lever. Speculator [23] and Xsyncfs [24] met this wall two decades ago with considerable kernel machinery and drew the same line: they buffered output until it was safe, and never claimed to un-output.

1.3 The observation

The claim conflates two distinct events.



Compensating transactions, sagas and idempotency keys all begin downstream of emission, and are best-effort by construction because they operate where the impossibility genuinely binds. The interval between the system call and emission — wholly inside the kernel — is unoccupied, and is the only place a *guarantee* rather than a best effort is available.

We note that this differs from idempotency keys in kind, not merely in degree: an idempotency key requires the *receiver* to implement it, whereas deferral requires nothing of the receiver at all.

1.4 Scope: what the mechanism does not reach

Not every agent incident is within reach, and we state the boundary early because a motivating example that the mechanism does not address is worse than no example. In 2026 an agent deleted a production database and its backups in nine seconds [12]. AgentTx’s *classifier* would not have caught it: each individual operation was routine, and per-call classification is by construction a local decision. AgentTx’s *deferral* would not have held it either, since a remote database mutation is a request–response exchange in which the agent blocks on the reply (§3.5).

What AgentTx offers in that scenario is the *transaction*: a destructive sequence staged under a verification-delimited boundary whose commit gate is “the test suite still passes” never commits, regardless of how routine each step appeared. Mechanism and motivating example must be matched, and we distinguish throughout which of our contributions addresses which failure.

1.5 Contributions

- Kernel-level deferral of fire-and-forget external effects.** Outbound operations issued inside a transaction are captured in a kernel write-ahead log and released only on commit. For this class, abort provides a hard guarantee where the state of the art provides best-effort compensation.
- Verification-delimited transaction boundaries.** We argue that the boundary question is the governing design decision, propose an answer derived from when evidence becomes available, and measure the resulting trade-off between guarantee window, snapshot staleness and log growth (§4).
- In-kernel effect classification.** An `int8`-quantised classifier executes with integer arithmetic inside an eBPF LSM hook, removing the per-call context switch that makes `seccomp` user-notify mediation too expensive to leave enabled.
- A four-class effect taxonomy**, grounded in the irrevocable transaction literature, with the first measured distribution of real agent system-call traces across it.
- A comparative evaluation** of agent rollback mechanisms under one workload and one metric set.

1.6 What we do not claim

We do not claim the transaction abstraction, which is TxOS [27]; output buffering, which is Xsyncfs [24]; fixed-point inference under the eBPF verifier, which is established practice [1, 2, 19]; filesystem rollback for agents, which ships in production and appears in concurrent work [11, 5, 15]; or the ability to reverse observed effects, which is impossible.

We further decline to claim automatic inference of compensation windows (§3.4), which we believe the kernel cannot obtain and which we name as an open problem rather than a contribution.

2 Background and Related Work

2.1 Operating system transactions

TxOS [27] added `sys_xbegin`, `sys_xend` and `sys_xabort` to Linux 2.6.22, providing ACID semantics for OS resources using lazy version management over shadow copies of kernel objects. It cost 3,300 lines for transaction management and 5,300 for object management, plus roughly 14,000 lines of changes to existing kernel code; a transactional OpenSSH installation incurred 10% overhead. It was never merged.

Its coverage is the relevant fact for us. TxOS provided transactional semantics for **150 of 303 Linux system calls**, with 153 explicitly unsupported ([27, Table 1]). `socket` is among the unsupported communication calls, alongside `ipc`, `mq_open` and `mq_unlink`; the authors note that “extending transactions to include networking (a real challenge) would increase the count of supported calls by 5.” On encountering an unsupported call, “the system logs a warning or aborts the transaction, depending on the flags passed to `sys_xbegin()`” ([27, §3.2]). There is no irrevocable-transaction mechanism: un-undoable work is handled by exclusion, then by warning or abort — never by committing early under a no-abort guarantee. Nor does TxOS restore application state on abort, only kernel state ([27, §3.1.4]).

Most directly relevant, TxOS anticipated our central limit. Its §3.1.5 observes that “code that communicates outside of a transaction and requires a response cannot be encapsulated into a single system transaction,” that “the system might buffer the message until commit,” and that a transaction awaiting a reply on the same channel will deadlock. Its resolution: “*the programmer is responsible* for avoiding this send/reply idiom within a transaction.”

TxFs [21, 22] subsequently showed that ACID filesystem transactions are obtainable in 5,200 lines by reusing the ext4 journal.

Our position. The abstraction is inherited, and so — we discovered on reading it closely — is the observation that request–response communication cannot be transactional and that outbound messages may be buffered until commit. What does not transfer is TxOS’s resolution. **In the agent setting there is no programmer** to be held responsible: the sequence of system calls is generated at run time, non-deterministically, by a process under adversarial influence. An escape hatch that reads as reasonable engineering advice in 2009 is precisely the assumption that fails in 2026. Our contribution is therefore not the idea of buffering but its enforcement without a cooperating programmer: classification of which effects may be buffered, at the network layer TxOS excluded,

with commit authority removed from the party issuing the calls (§4.2).

2.2 Irrevocability: the correct frame

The problem of I/O inside a transaction is long-standing and named. The software transactional memory literature calls a transaction performing un-undoable work *irrevocable* or *inevitable*, and resolves it not by reversing the I/O but by guaranteeing in advance that the transaction will never abort, typically by serialising it so that it wins all conflicts [29, 28].

We adopt this frame, with one caveat we must confront rather than inherit: that literature assumes short-lived transactions bounded by a few thousand instructions. Agent transactions are bounded by human-scale activities lasting seconds to minutes, and the assumption does not transfer. §4.1 treats this as a first-class problem.

2.3 Speculation and external output

Speculator [23] provided kernel support for speculative execution, checkpointing processes and propagating causal dependencies through IPC. Xsyncfs [24] developed *external synchrony*: buffer externally observable output until the state it depends on is durable.

Our position. Our effect log is external synchrony with a different trigger — we delay until the enclosing transaction commits rather than until state is durable — applied to network sends rather than file output, in a setting where the emitting process is untrusted rather than merely impatient. That last difference is not cosmetic: it forces a threat model (§4.4) that neither prior system required.

2.4 Agent sandboxing, rollback and state

Concurrent work presents a transactional sandbox for coding agents combining policy interception with filesystem snapshots, reporting complete interception of high-risk commands and complete rollback at roughly 14.5% overhead [5]. DeltaBox [15] achieves millisecond checkpoint and rollback through a change-based state abstraction. Cordon [13] places transactions at the tool-call boundary. AgentSight [4] demonstrates eBPF observability at under 3% overhead by correlating intercepted model traffic with kernel events — a technique we adopt for payload inspection (§3.5). ACRFence [6] studies attacks against agent checkpoint-restore itself. ChronoMem [10] and MemTX [16] apply versioning and transactional commit to agent *memory*, which we draw on in §4.3. ActPlane [7] and CRAB [14] are reported to perform in-kernel eBPF policy enforcement and effect classification respectively. **[TODO: Read [7] and [14] in full before the design freeze. If either already defers external effects in-kernel, Contribution 1 collapses and the project repositions onto Contributions 2 and 5.]**

Orthogonally, CaMeL [20] defeats prompt injection by construction. AgentTx is complementary: CaMeL constrains what an agent decides; AgentTx constrains what its system calls can make real.

2.5 Machine learning inside the kernel

KML [19, 18] demonstrated in-kernel neural networks and decision trees in under 4KB of dynamic memory at below 0.2% CPU overhead. Within eBPF the verifier forbids floating point; the established response is to train in user space, quantise to integer parameters, ship weights through BPF maps and execute layers via tail calls, with in-kernel inference roughly 84% faster than the user-space equivalent [1, 2, 3].

3 Mechanism

3.1 Interface

```
tx_begin(flags) tx_commit() tx_abort()
```

During development these are `ioctl`s on a character device; the final implementation promotes them to system calls. Critically, `tx_commit` is *not* available to the transacting process (§4.2).

3.2 Isolation model

On `tx_begin` the transaction manager mounts an overlay in a private mount namespace belonging to the transacting process: the working directory as read-only lower layer, a per-transaction staging directory as upper, merged at the original path and visible only inside that namespace.

The agent observes an unchanged path and reads its own writes. Every other process — the user’s editor, shell and version control — observes the unmodified directory, yielding snapshot isolation and ensuring an abort cannot invalidate an open editor buffer. Storage cost is proportional to the set of files written, not the size of the working directory; where the underlying filesystem supports cloning, copy-up may be satisfied by a reflink. We measure both cases.

3.3 The four-class taxonomy

eBPF LSM programs attach to `file_open`, `inode_unlink`, `inode_rename`, `socket_connect`, `socket_sendmsg` and `bprm_check_security`. Each intercepted operation is classified:

Class	Handling
Reversible	proceed; the CoW layer captures it
Deferrable	record in the effect WAL, report success, do not emit
Compensable	emit, record a cancellation window and commit deadline
Irrevocable	the transaction becomes non-abortable and terminates; escalate to a human approval gate, deny on timeout

Commit flushes the WAL in issue order and merges the upper layer down; abort discards both. A write to a terminal has already been observed by a human and is irrevocable by construction.

3.4 The compensable class: a registry, not an inference

Some destinations do cooperate: a payment may be voided before capture, a push force-reverted before anyone pulls, a resource deleted after creation. The kernel, however, has no application semantics with which to discover this. We considered and rejected inferring cancellation windows from traffic; nothing observable at the LSM hook distinguishes an endpoint that honours a void from one that does not.

We therefore specify the compensable class as a *declarative registry*, distributed and updated like a certificate bundle: a mapping from host and path pattern to a cancellation endpoint and a window duration. The kernel consults it; it does not learn it. A transaction that emits a compensable effect acquires a commit deadline equal to the shortest window it holds, and is aborted — with compensation attempted — if the deadline passes.

This is deliberately modest. Automatic inference of compensation semantics is, we believe, not solvable at the kernel boundary, and we name it as an open problem rather than claim it (§1.6).

3.5 The request–response limit

Deferral is possible only where the agent does not block on a reply. A fire-and-forget effect — a push, a webhook, a notification, a payment submission — may be held; a request–response exchange such as an API query or package download may not, and a transaction that waits on a buffered channel deadlocks outright.

This limit is not ours to discover: TxOS states it precisely in its §3.1.5 and resolves it by assigning the programmer responsibility for avoiding the send/reply idiom inside a transaction. We inherit the limit and reject the resolution, because no programmer is present to be held responsible. The consequence is that what TxOS could leave as a documented caveat, we must detect and enforce at run time: an effect that would block must be recognised as such before it is buffered, or the agent deadlocks.

Classification therefore does work in AgentTx that it did not have to do in TxOS.

The magnitude of Contribution 1 is bounded by the fraction of agent outbound effects that are fire-and-forget, a quantity nobody has measured. We treat this as a first-class result and a project gate (§6).

A second limit is visibility: at the syscall boundary a TLS `sendmsg` carries opaque bytes, so mutating and non-mutating operations to the same host are indistinguishable. We adopt AgentSight’s approach [4], attaching uprobes to the TLS library write path to recover plaintext. This is precisely where AgentTx improves on destination allowlists, which are static and per-host and cannot separate a read from a mutation against the same endpoint.

3.6 In-kernel classification

A small model is trained in user space on labelled agent traces, quantised to `int8` and loaded into a BPF map; the forward pass uses integer arithmetic and a fixed-point sigmoid lookup table with bounded iteration. Features are restricted to quantities cheaply computable in hook context: syscall number, hashed path prefix, descriptor type, destination port, transaction depth and a short call n-gram. The classifier is *fail-closed*: an unknown or low-confidence result is treated as irrevocable and escalated, never silently permitted (§4.4).

4 Policy

A deferral mechanism is only as meaningful as the rule governing release. This section addresses the questions the mechanism alone leaves open.

4.1 What is a transaction?

Nothing in §3 says how long a transaction lasts, and the answer determines whether the guarantee is worth anything.

Boundary	Consequence
Per tool call	Guarantee window of milliseconds. Effects commit almost immediately; protection is near-nil
Per verification	Window equal to an edit-build-test cycle. Bounded staleness, bounded log
Per task or session	Large window, but unbounded log across an unbounded number of agent decisions, and a snapshot arbitrarily stale

The question is not new but has never been answered. TxOS raised it and deliberately left it open, observing that “allowing a long running transaction to isolate all

system resources indefinitely is undesirable, so the OS may want a policy that limits the size of a transaction,” and comparing the matter to controlling any process that abuses system resources ([27, §3.1.3]). It supplied the mechanism and declined the policy. For a workload whose transactions are bounded by human-scale activity rather than by a critical section, the policy can no longer be deferred.

We argue for the middle option on principle rather than convenience. The reason to speculate at all is that safety cannot be decided in advance; the moment evidence *does* arrive — the build succeeds, the test suite passes, a human reviews the diff — the decision should be taken. A transaction should therefore span exactly the interval over which no evidence is available. We call this a *verification-delimited* transaction. Its length is determined by the workload rather than configured, and a session becomes a sequence of transactions rather than one long one.

Two properties follow. First, the taxonomy and the boundary interlock: an irrevocable effect terminates the enclosing transaction, exactly as an irrevocable operation forces commitment in STM [29]. Second, because agent transactions are orders of magnitude longer than those the STM literature assumes, its results on serialising irrevocable transactions cannot be imported unmodified; we expect contention behaviour to differ and will report it.

We do not assert that verification-delimited is optimal. We make transaction length an evaluation axis (§6) and report guarantee window, staleness-induced conflict rate and log growth at each of the three boundaries.

4.2 Who may commit

The agent is the untrusted party. It must therefore not hold commit authority: `tx_commit` invoked by a process inside the transaction returns `-EPERM`. Commit is exercised only by the supervising process, which is outside the transaction and is driven by the verification signal, a policy rule, or a human.

This single invariant eliminates the most obvious attack on the system — a prompt-injected agent instructed to commit before its exfiltration is inspected — and it is why `tx_begin` and `tx_commit` are asymmetric in a way TxOS’s interface was not.

4.3 Coherence of agent reasoning

Reporting success for a deferred send makes the agent’s subsequent reasoning conditional on an event that may never occur. If the transaction later aborts, the agent holds a chain of inferences built on a false premise.

TxOS faced the analogous problem and left it to the caller: on abort “the OS restores the kernel state to its pre-transaction state, but it does not revert application state” ([27, §3.1.4]), offering an optional copy-on-write checkpoint of the application address space, in the manner of Speculator, as a flag to `sys_xbegin`. For an agent

this cannot be optional, and the address space is the wrong granularity in any case: the state that must be reverted is the conversation, which lives in the supervising harness rather than in the transacting process.

We therefore place the agent’s reasoning state *inside* the transaction. Conversation and scratch state are owned by the supervising harness, staged in the same copy-on-write layer as the filesystem, and discarded on abort along with everything else. An abort therefore returns the agent to the epistemic position it occupied at `tx.begin`, with no false premise surviving. This mirrors the versioned and transactional treatment of agent memory developed in [10, 16], and it is the reason our transaction spans three kinds of state rather than two.

We considered the alternative of returning an explicit DEFERRED status so the agent may reason about pending effects. It requires agent cooperation and a model that handles the concept correctly, and we regard it as future work.

4.4 Threat model

The adversary is a prompt injection that fully controls the agent’s outputs. It does not hold a kernel exploit, does not control the supervising harness, and does not run as root. Attacks on AgentTx itself and our responses:

Attack	Response
Premature commit	Agent cannot commit (§4.2)
Escape the transaction by spawning outside the namespace	Namespace is inherited across <code>fork/execve</code> ; <code>unshare</code> and <code>setns</code> denied inside a transaction
Adversarial evasion of the classifier	Fail-closed: low confidence escalates rather than permits. Evasion yields a prompt, not a silent pass
WAL exhaustion (emit millions of effects to force a flush)	Bounded log; overflow aborts the transaction rather than releasing it
TOCTTOU between classification and emission	Classification and disposition occur in the same hook invocation
Bypass TLS uprobes via static linking or custom crypto	Degrades to destination-only classification; an acknowledged coverage gap
Escape via uninstrumented paths	The general weakness of interception designs [17]; we enumerate covered hooks explicitly rather than claim completeness

Out of scope: kernel exploits, a compromised supervisor, malicious agent binaries as opposed to injected agent *instructions*, and timing side channels — deferral itself perturbs observable timing, which we note but do not defend against.

5 Implementation Plan

The system decomposes into four components with interfaces frozen in week two and one developer each. Every component ships a stub implementation on day one, so any developer can build and boot the complete system in isolation and no component is blocked on another’s progress.

	Component	Kernel surface
P1	Transaction core	syscall table, <code>task_struct</code> , locking
P2	CoW storage	VFS, overlays, stackable FS
P3	Effect interception	BPF LSM, networking, ring buffers
P4	Policy, evaluation	verifier, quantisation, measurement

Development proceeds in four independently demonstrable rungs: a loadable module with `ioctl` control and overlay-backed abort; LSM interception with a static rule table; replacement of that table by the in-kernel classifier; and promotion to real system calls. Should a rung prove intractable, the preceding rung remains a complete system. Multi-agent optimistic concurrency control is scoped as a stretch objective.

6 Evaluation Plan

Project gate. In weeks three to four we instrument real agent sessions and measure the fraction of outbound network operations that are fire-and-forget rather than request–response. Above 20%, Contribution 1 stands and the measurement becomes a headline figure. Below 10%, we narrow the claim and promote Contributions 2 and 5. This costs four days and de-risks the remainder of the project.

Boundary sweep. We run the full workload at each of the three transaction boundaries of §4.1, reporting guarantee window, staleness-induced conflict rate at commit, peak WAL size, and the fraction of transactions terminated early by an irrevocable effect. This converts the central design decision from an assertion into a measurement.

Metrics. Task success rate; containment rate under adversarial input; interception latency; end-to-end overhead; storage amplification; classifier accuracy before and after quantisation, and under adversarial perturbation; and the empirical distribution of agent effects across the taxonomy.

Baselines. Unprotected execution; session checkpoints as shipped in production agents [11]; `git` checkpoint and reset; container rebuild; microVM snapshot; static `seccomp` allowlist; user-space policy daemon via `seccomp` user-notify; and, where artifacts permit, [5] and [15], whose published overheads of 14.5% and under 3% are the figures we must meet or explain.

Workloads. A benign corpus of realistic development tasks; a fault-injection corpus of destructive actions; and an adversarial corpus in which prompt injections are planted in files the agent reads during ordinary operation, including the exfiltration scenario of §1.

Headline figures. (1) The deferrable fraction of agent outbound effects. (2) Guarantee window against staleness across transaction boundaries. (3) In-kernel classification latency against accuracy.

7 Limitations

Terminal output cannot be staged. Deferring a send means reporting success for an operation that has not occurred; §4.3 addresses the consequences for agent reasoning but not for a program that inspects the network result directly. Coverage is the weakest point of any interception design. Rollback mechanisms are themselves an attack surface [6]. Our classifier is trained on traces from a specific set of agents and may not transfer. The compensable class depends on a registry somebody must maintain. And request–response effects, which may prove to be the majority, are outside the mechanism entirely.

8 Conclusion

System transactions were proposed in 2009 and set aside for want of a workload justifying their cost. Autonomous agents supply that workload. Filesystem rollback for agents is now solved and we do not contest it; what remains is the class of effects the field has conceded as irreversible. We argue that class has been drawn too broadly, because irreversibility is a property of observation rather than of the system call, and that the kernel occupies the one interval in which a guarantee is still available. Equally, we argue that the mechanism is inert without an answer to what a transaction *is* for an agent, and we propose to answer that question by measurement rather than assertion.

References

- [1] Dynamic fixed-point values in eBPF: A case for fully in-kernel anomaly detection. In *Proceedings of the ACM SIGCOMM Workshop on eBPF and Kernel Extensions*, 2024. VERIFY authors. The fixed-point technique AgentTx’s in-kernel classifier depends on.
- [2] Practicality of in-kernel/user-space packet processing empowered by lightweight neural network and decision tree. *Computer Networks*, 2024. VERIFY authors and DOI. int8 quantisation, weights in BPF maps, per-layer tail calls, 84% lower inference latency than user space.
- [3] When eBPF meets machine learning: On-the-fly OS kernel compartmentalization. arXiv:2401.05641, 2024. VERIFY authors.
- [4] AgentSight: System-level observability for AI agents using eBPF. arXiv:2508.02736, 2025. Correlates TLS-intercepted LLM traffic with kernel events, under 3% overhead. VERIFY authors.
- [5] Fault-tolerant sandboxing for AI coding agents: A transactional approach to safe autonomous execution. arXiv:2512.12806, 2025. CLOSEST PRIOR WORK. Policy-based interception plus transactional FS snapshots; reports 100% interception of high-risk commands, 100% rollback success, 14.5% overhead. VERIFY authors and venue.
- [6] ACRFence: Preventing semantic rollback attacks in agent checkpoint-restore. arXiv:2603.20625, 2026. Attacks against rollback itself — read before you claim your rollback is safe. VERIFY.
- [7] ActPlane: In-kernel agent action policy enforcement with eBPF and information-flow control, 2026. CITATION INCOMPLETE. Referenced in secondary sources only; locate the primary paper before citing. Overlaps AgentTx’s P3 workstream.
- [8] Agent rollback and checkpoint patterns: A 2026 engineering reference. Industry reference, 2026. Three reversibility tiers: checkpoints, sagas, idempotency keys. All application-layer — the gap AgentTx targets. Source of the tier-3 impossibility claim we quote.
- [9] ChatGPT work: A production agent control plane. Industry analysis, 2026. Plan mode, configurable check-ins, action approvals. Irreversible actions gated on a human in the tool implementation.
- [10] ChronoMem: Version control and semantic rollback for large language model agent memory. arXiv:2607.27773, 2026. VERIFY.
- [11] Claude code checkpoints and rewind. Product documentation, 2026. Checkpoints recorded before every filesystem-modifying action; rewind restores to any prior tool call. Evidence that filesystem rollback is commodity.

- [12] Coding agent wipes production database and backups in nine seconds. Press report, 2026. Motivating incident. VERIFY details independently before citing — press reports of such incidents are frequently inaccurate in the specifics.
- [13] Cordon: Semantic transactions for tool-using LLM agents. arXiv:2606.17573, 2026. Transactions at the tool-call boundary rather than the syscall boundary. VERIFY authors.
- [14] CRAB: Aligning durable checkpoints with agent turn boundaries via eBPF effect classification, 2026. CITATION INCOMPLETE. Referenced in secondary sources only; locate the primary paper before citing. Overlaps AgentTx’s effect-classification contribution.
- [15] DeltaBox: Scaling stateful AI agents with millisecond-level sandbox checkpoint/rollback. arXiv:2605.22781, 2026. DeltaState abstraction; DeltaFS is OverlayFS-inspired change-based FS checkpoint/restore. VERIFY authors.
- [16] MemTX: Transactional belief commit for stateful agent memory. arXiv:2607.23929, 2026. VERIFY.
- [17] Your coding agent sandbox probably leaks. here is where. Industry analysis, 2026. Symlink and work-tree escapes; motivates why coverage is the weak point of any interception design.
- [18] Ibrahim Umit Akgun, Ali Selman Aydin, Andrew Burford, Michael McNeill, Michael Arkhangelskiy, and Erez Zadok. KML: Using machine learning to improve storage systems. arXiv:2111.11554, 2021.
- [19] Ibrahim Umit Akgun, Ali Selman Aydin, Andrew Burford, Michael McNeill, Michael Arkhangelskiy, and Erez Zadok. Improving storage systems using machine learning. *ACM Transactions on Storage*, 2023. KML. In-kernel neural nets and decision trees in under 4 KB of kernel memory, CPU overhead below 0.2%.
- [20] Edoardo Debenedetti et al. Defeating prompt injections by design. arXiv:2503.18813, 2025. CaMeL. Capability-based control/data-flow separation for tool-using agents; 77% of AgentDojo tasks with provable security. VERIFY full author list.
- [21] Yige Hu, Zhiting Zhu, Ian Neal, Youngjin Kwon, Tianyu Cheng, Vijay Chidambaram, and Emmett Witchel. TxFS: Leveraging file-system crash consistency to provide ACID transactions. In *Proceedings of the 2018 USENIX Annual Technical Conference (USENIX ATC ’18)*, 2018. Best Paper. Reuses the ext4 journal for ACID file-system transactions; 5,200 LoC.
- [22] Yige Hu, Zhiting Zhu, Ian Neal, Youngjin Kwon, Tianyu Cheng, Vijay Chidambaram, and Emmett Witchel. TxFS: Leveraging file-system crash consistency to provide ACID transactions. *ACM Transactions on Storage*, 15(2), 2019. Extended journal version of [21].
- [23] Edmund B. Nightingale, Peter M. Chen, and Jason Flinn. Speculative execution in a distributed file system. In *Proceedings of the 20th ACM Symposium on Operating Systems Principles (SOSP ’05)*. ACM, 2005. Best Paper. Kernel support for speculation, checkpoint/rollback, causal dependency tracking across IPC.
- [24] Edmund B. Nightingale, Kaushik Veeraraghavan, Peter M. Chen, and Jason Flinn. Rethink the sync. In *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI ’06)*, 2006. Best Paper. External synchrony: buffer externally visible output until the state it depends on is durable. Direct ancestor of AgentTx’s deferred-effect log.
- [25] OpenAI. Agent approvals and security. Product documentation, 2026.
- [26] OpenAI. Codex CLI sandboxing implementation. Project documentation, 2026. bubblewrap + unprivileged user namespaces + Landlock LSM + seccomp-BPF on Linux; Seatbelt on macOS; restricted tokens and synthetic SIDs on Windows. Default-deny egress with allowlists and DNS/IP classification.
- [27] Donald E. Porter, Owen S. Hofmann, Christopher J. Rossbach, Alexander Benn, and Emmett Witchel. Operating system transactions. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP ’09)*, Big Sky, MT, USA, 2009. ACM. TxOS. VERIFIED against the paper 2026-08-04. 150 of 303 syscalls transactional (Table 1); socket/ipc/mq_{open}unsupported; unsupportedcalllogsawarningon transactionmechanism; applicationstatenotrevertedonabort(§3. responsecommunicationidentifiedasnon — transactionalwithbufferingsuggestedandresponsibilityassigned runningtransactionsizingexplicitlyleftasopenpolicy(§3.1.3).
- [28] Michael F. Spear, Michael Silverman, Luke Dalessandro, Maged M. Michael, and Michael L. Scott. Implementing and exploiting inevitability in software transactional memory. In *Proceedings of the 37th International Conference on Parallel Processing (ICPP ’08)*, 2008. VERIFY venue/year and author list.
- [29] Adam Welc, Bratin Saha, and Ali-Reza Adl-Tabatabai. Irrevocable transactions and their applications. In *Proceedings of the 20th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA ’08)*, 2008. VERIFY venue/year. A transaction performing un-undoable

work becomes irrevocable: the system guarantees it will not abort rather than attempting to reverse the I/O.